

EduSmart

Plateforme éducative intelligente

Rapport de la semaine 2

Prototypage backend en architecture microservices

Réalisé par : Alpha B

Technologies : Node.js, Express.js, PostgreSQL, MongoDB, Docker, API Gateway, JWT

Date : 14 juillet 2026

Sommaire

1. 1. Introduction
2. 2. Objectifs de la semaine 2
3. 3. Architecture microservices retenue
4. 4. Services développés
5. 5. Configuration Docker et bases de données
6. 6. Sécurité et authentification JWT
7. 7. Routes API développées
8. 8. Scénario de test validé
9. 9. Problèmes rencontrés et corrections
10. 10. Bilan de la semaine 2
11. 11. Prochaine étape
12. Annexes

1. Introduction

La deuxième semaine du projet EduSmart a été consacrée à la mise en place du prototype backend. Le choix technique principal est l'utilisation d'une architecture microservices, conformément au cahier des charges. L'objectif est de séparer les grandes responsabilités de la plateforme afin de faciliter la maintenance, l'évolution et les tests indépendants de chaque module.

Le prototype backend couvre le scénario principal suivant : un étudiant se connecte, consulte un cours, s'y inscrit, enregistre sa progression, obtient un score de quiz, puis reçoit une recommandation personnalisée selon son niveau et ses résultats.

2. Objectifs de la semaine 2

- Créer la structure backend en microservices.
- Configurer une API Gateway comme point d'entrée unique.
- Mettre en place le service d'authentification avec JWT.
- Connecter PostgreSQL pour les données structurées.
- Connecter MongoDB local avec Docker pour les cours et les interactions.
- Créer les routes principales de chaque microservice.
- Tester le scénario complet : inscription, connexion, cours, progression, quiz et recommandation.

3. Architecture microservices retenue

Le frontend communique uniquement avec l'API Gateway. Celle-ci redirige ensuite les requêtes vers les services concernés. Chaque microservice possède une responsabilité fonctionnelle claire. Cette séparation permet d'éviter un backend monolithique et prépare le projet à une évolution plus professionnelle.

EduSmart - Architecture backend microservices

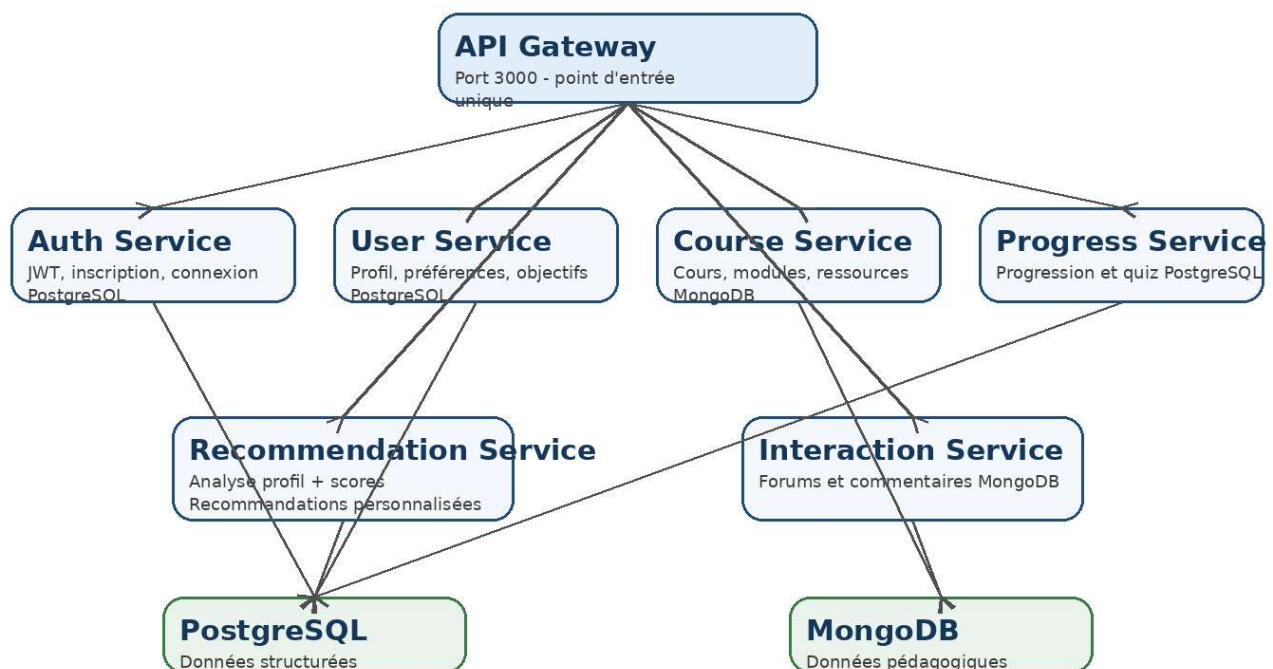


Figure 1 : Architecture backend microservices de la plateforme EduSmart.

Service	Port	Responsabilité	Base utilisée
API Gateway	3000	Point d'entrée unique vers les microservices	Aucune
Auth Service	4001	Inscription, connexion, JWT et rôles	PostgreSQL
User Service	4002	Profils, préférences et objectifs	PostgreSQL
Course Service	4003	Cours, modules, ressources et quiz	MongoDB
Progress Service	4004	Inscriptions, progression, scores de quiz	PostgreSQL
Recommendation Service	4005	Génération des recommandations personnalisées	PostgreSQL + appels API
Interaction Service	4006	Forums et commentaires	MongoDB

4. Services développés

4.1 API Gateway

L'API Gateway reçoit toutes les requêtes du frontend. Elle masque les ports internes des microservices et permet au client d'utiliser une seule URL de base.

```
GET /health
POST /api/auth/register -> Auth Service
POST /api/auth/login -> Auth Service
GET /api/users/profile -> User Service
GET /api/courses -> Course Service
POST /api/progress/enrollments -> Progress Service
POST /api/recommendations/generate -> Recommendation Service
```

4.2 Auth Service

Le service d'authentification gère l'inscription, la connexion et la génération du token JWT. Les mots de passe sont hachés avant l'enregistrement. Les rôles supportés sont ADMIN, INSTRUCTOR et STUDENT.

4.3 User Service

Le User Service gère le profil de l'étudiant, ses préférences d'apprentissage et ses objectifs. Ces données sont utilisées par le Recommendation Service pour personnaliser les suggestions.

4.4 Course Service

Le Course Service stocke les cours dans MongoDB. Cette base NoSQL est adaptée aux cours, car les modules, ressources, vidéos, PDF, articles et quiz peuvent avoir des structures différentes.

4.5 Progress Service

Le Progress Service enregistre les inscriptions aux cours, la progression sur les ressources et les résultats de quiz. Ces informations structurées sont stockées dans PostgreSQL.

4.6 Recommendation Service

Le Recommendation Service est responsable de la logique intelligente du projet. Dans ce prototype, il utilise une approche basée sur des règles : si le score de l'étudiant est faible, le système recommande des ressources de renforcement ; si le score est élevé, il propose des contenus plus avancés.

4.7 Interaction Service

Le Interaction Service prépare les fonctionnalités sociales de la plateforme : forums, publications de discussion et commentaires liés aux cours.

5. Configuration Docker et bases de données

Le projet est lancé avec Docker Compose. Cette configuration permet de démarrer automatiquement les microservices, PostgreSQL et MongoDB dans des conteneurs séparés.

```
docker compose up --build
```

Pour cette semaine, MongoDB est testé localement dans Docker. La connexion Atlas n'est pas encore utilisée afin de valider d'abord l'architecture en local.

Base de données	Utilisation	Services concernés
PostgreSQL	Utilisateurs, rôles, profils, progression, recommandations	Auth, User, Progress, Recommendation
MongoDB	Cours, modules, ressources, forums, commentaires	Course, Interaction

6. Sécurité et authentification JWT

Les routes sensibles sont protégées par un middleware JWT. Après connexion, l'utilisateur reçoit un token. Ce token doit être envoyé dans les en-têtes HTTP des requêtes protégées.

```
Authorization: Bearer TOKEN_UTILISATEUR
```

Le contrôle des rôles est utilisé pour limiter certaines actions. Par exemple, seuls les formateurs ou administrateurs peuvent créer et publier un cours.

7. Routes API développées

Module	Méthode	Route	Description
Auth	POST	/api/auth/register	Créer un compte utilisateur
Auth	POST	/api/auth/login	Connecter un utilisateur
User	GET	/api/users/profile	Consulter le profil connecté
User	PUT	/api/users/profile/student	Modifier le profil étudiant
Course	POST	/api/courses	Créer un cours
Course	POST	/api/courses/:id/modules	Ajouter un module
Course	POST	/api/courses/:courseId/modules/:moduleId/ressources	Ajouter une ressource
Course	PATCH	/api/courses/:id/publish	Publier un cours
Course	GET	/api/courses	Lister les cours publiés
Progress	POST	/api/progress/enrollments	Inscrire un étudiant à un cours
Progress	PUT	/api/progress/lessons	Créer ou mettre à jour la progression
Progress	POST	/api/progress/quiz-attempts	Enregistrer un résultat de quiz
Recommendation	POST	/api/recommendations/generate	Générer des recommandations
Recommendation	GET	/api/recommendations/me	Afficher les recommandations
Recommendation	POST	/api/recommendations/:id/feedback	Ajouter un feedback

8. Scénario de test validé

Le scénario principal validé pendant les tests est le suivant : création d'un étudiant, création d'un formateur, création d'un cours, publication du cours, inscription de l'étudiant, enregistrement d'un score de quiz, puis génération d'une recommandation.

Étape	Action testée	Résultat attendu
1	GET /health sur API Gateway	Service disponible
2	Inscription étudiant	Utilisateur créé avec rôle STUDENT
3	Connexion étudiant	Token JWT retourné
4	Inscription formateur	Utilisateur créé avec rôle INSTRUCTOR
5	Création d'un cours	Cours créé dans MongoDB
6	Ajout module et ressource	Structure pédagogique enrichie

7	Publication du cours	Cours visible par les étudiants
8	Inscription de l'étudiant au cours	Progression initialisée
9	Score de quiz faible	Score enregistré dans PostgreSQL
10	Génération de recommandation	Cours de renforcement recommandé

8.1 Exemple de recommandation obtenue

Après un score faible au quiz, le système a généré une recommandation personnalisée. Le résultat confirme que le Recommendation Service utilise les scores pour adapter les contenus proposés.

```
{
  "message": "Recommandations générées",
  "recommendations": [
    {
      "recommendation_type": "COURSE",
      "reason": "Score faible : ressources de renforcement recommandées.",
      "recommendation_score": "85.00",
      "status": "PENDING",
      "courseTitle": "Introduction à JavaScript"
    }
  ]
}
```

9. Problèmes rencontrés et corrections

Problème	Cause	Correction appliquée
Docker non reconnu	Docker Desktop non installé ou intégration WSL non activée	Installation de Docker Desktop et activation WSL 2
Cannot POST /register	Mauvaise redirection dans API Gateway	Correction du proxy pour rediriger vers /auth/register
Identifiants incorrects	Utilisateur ou mot de passe différent de l'inscription	Création d'un nouvel utilisateur de test et vérification PostgreSQL
Cannot POST /progress/lessons	La route était définie en PUT et non en POST	Utilisation correcte de PUT /api/progress/lessons

9.1 Correction importante dans l'API Gateway

La redirection de l'API Gateway a été corrigée afin que les requêtes publiques soient envoyées vers le préfixe réel du microservice.

```
const proxy = (publicPath, target, servicePath) => {
  app.use(
    publicPath,
    createProxyMiddleware({
      target: `${target}${servicePath}`,
      changeOrigin: true
    })
  );
};

proxy('/api/auth', process.env.AUTH_SERVICE_URL, '/auth');
proxy('/api/users', process.env.USER_SERVICE_URL, '/users');
proxy('/api/courses', process.env.COURSE_SERVICE_URL, '/courses');
proxy('/api/progress', process.env.PROGRESS_SERVICE_URL, '/progress');
```

10. Bilan de la semaine 2

La semaine 2 est validée, car le backend microservices est opérationnel. Les services principaux démarrent avec Docker, l'API Gateway fonctionne, les données PostgreSQL et MongoDB sont accessibles, et le scénario principal de recommandation a été testé avec succès.

Points validés :

- API Gateway opérationnelle.
- Auth Service fonctionnel avec JWT.
- Course Service connecté à MongoDB local Docker.
- Progress Service connecté à PostgreSQL.
- Recommendation Service capable de générer une recommandation selon le score.
- Interaction Service prêt pour les forums et commentaires.

11. Prochaine étape

La semaine suivante pourra être consacrée au développement du frontend React et à son intégration avec l'API Gateway. L'objectif sera de créer les pages d'inscription, connexion, tableau de bord étudiant, liste des cours, détail du cours, progression et recommandations.

Priorité	Travail prévu
1	Créer le frontend React avec Vite
2	Créer les pages Login et Register
3	Créer le tableau de bord étudiant
4	Afficher les cours publiés
5	Connecter les routes de progression et recommandations

Annexe A : commandes utiles

```
# Lancer tous les services
docker compose up --build

# Arrêter les services
docker compose down

# Reconstruire uniquement l'API Gateway sans cache
docker compose build --no-cache api-gateway

# Voir les conteneurs actifs
docker ps

# Ouvrir PostgreSQL dans Docker
docker compose exec postgres psql -U edusmart -d edusmart_db
```

Annexe B : exemples de requêtes de test

B.1 Inscription étudiant

POST http://localhost:3000/api/auth/register
Content-Type: application/json

```
{
  "firstName": "Alpha",
  "lastName": "barry",
  "email": "alpha@test.com",
  "password": "123456",
  "role": "STUDENT"
}
```

B.2 Création d'un cours

POST http://localhost:3000/api/courses
Authorization: Bearer TOKEN_FORMATEUR
Content-Type: application/json

```
{
  "title": "Introduction à JavaScript",
  "description": "Cours pour apprendre les bases de JavaScript.",
  "category": "Programmation Web",
  "level": "BEGINNER",
  "tags": ["javascript", "web", "frontend"]
}
```

B.3 Génération de recommandation

POST http://localhost:3000/api/recommendations/generate
Authorization: Bearer TOKEN_ETUDIANT

Fin du rapport de la semaine 2.